Prompt Documentation

Project Title

AI-Powered Ticket Management MCP Server

This document summarizes the key prompts used during the development, debugging, testing, and deployment of the project.

---

1. Master Project Development Prompt

Prompt:

Build a Ticket Management MCP Server using Python and SQLite. Implement MCP tools for ticket creation, retrieval, listing, commenting, searching, and lifecycle management. The system should support Claude Desktop integration and a custom Python client.

Outcome:

- MCP Server Architecture designed
- SQLite Database created
- Service Layer implemented
- MCP Tool Framework established

---

2. Database Design Prompt

Prompt:

Create a normalized SQLite schema for tickets, comments, and knowledge base articles. Include relationships, indexing, and search support.

Outcome:

- Tickets table created
- Comments table created
- Knowledge Base table created
- Database initialization scripts generated

---

3. MCP Tool Generation Prompt

Prompt:

Implement MCP tools with JSON schemas, handlers, and service integration for:

- create_ticket
- get_ticket
- list_tickets
- add_comment
- search_kb

Outcome:

Core MCP tools successfully implemented and exposed through the MCP server.

---

4. Ticket Lifecycle Enhancement Prompt

Prompt:

Extend the Ticket Management MCP Server with ticket lifecycle operations including status updates and ticket resolution.

Outcome:

Implemented:

- update_ticket_status
- resolve_ticket

---

5. Python Client Development Prompt

Prompt:

Create a Python client that consumes the MCP server and demonstrates ticket creation, retrieval, commenting, and searching.

Outcome:

Custom Python client implemented and tested successfully.

---

6. Claude Desktop Integration Prompt

Prompt:

Configure the MCP server to run with Claude Desktop using stdio transport and generate the required claude_desktop_config.json configuration.

Outcome:

Claude Desktop successfully connected and consumed MCP tools.

---

7. Debugging – Module Import Error

Issue

ModuleNotFoundError: No module named 'src'

Correction Prompt

Analyze the project structure and provide the correct execution method and working directory for the MCP server.

Resolution
Changed execution from:
python src/server/mcp_server.py
to:
cd ai-ticket-mcp
python -m src.server.mcp_server
Server started successfully.

---

8. Debugging – Claude Desktop MCP Connection Failure

Issue

Server disconnected. Could not attach to MCP server.

Correction Prompt

Verify MCP server startup command, working directory, Python path, and Claude Desktop configuration. Generate corrected claude_desktop_config.json settings.

Resolution

- Fixed command configuration
- Added correct working directory
- Corrected module execution method

Claude Desktop successfully detected the MCP server.

---

9. Debugging – Ticket Status Not Updating Correctly

Issue

Tickets were not properly moving to the Closed state during lifecycle testing.

Correction Prompt

Analyze ticket lifecycle workflow and validate status transition logic. Ensure ticket status changes are persisted correctly in SQLite and reflected through MCP tools.

Resolution

- Fixed lifecycle transition logic
- Updated database persistence layer
- Corrected MCP tool handling
- Added lifecycle validation

Final Workflow

Open → In Progress → Resolved → Closed

Lifecycle flow validated successfully.

---

10. Debugging – Missing Sample Data Documentation

Issue

Sample input/output files existed for only a subset of implemented MCP tools.

Correction Prompt

Analyze all implemented MCP tools and generate missing sample input and output JSON files matching the actual tool schemas.

Resolution

Generated missing sample datasets for:

- update_ticket_status
- resolve_ticket

Validated against implemented tool schemas.

---

11. Documentation Generation Prompt

Prompt:

Generate project documentation including README, architecture overview, setup guide, AI usage note, prompt documentation, and sample data documentation.

Outcome:

Generated:

- README.md
- AI_USAGE.md
- Prompt_Documentation.md
- Sample Data Documentation
- Project Setup Guide

Conclusion

AI tools including Claude, Gemini Antigravity, and ChatGPT were used throughout the project lifecycle for architecture design, code generation, debugging, testing, documentation, and deployment support. All AI-generated outputs were manually reviewed, corrected, validated, and tested by the project team before final submission.